Security Self-Assessment Report

CashPilot: an AI financial assistant for small businesses · Oct 20, 2026 · @Arnob

Contents: (1) permission-tier implementation, (2) known-risk list, (3) verification evidence.

> **Scope and method.** This is a self-assessment by the development team, based on reading the implementation, 264 automated tests, and a scripted sandbox run against the real API (see section 3). It is not an independent penetration test and makes no claim of certification. Items marked **planned** are not built.

1. Permission-tier implementation

Access is decided in five layers. A request must pass all of them; each layer is a separate piece of code, so one mistake does not open everything.

#	Layer	Implementation	Where
1	Authentication	Bearer JWT (15-minute lifetime) checked on every protected route; refresh token rotates and can be revoked	`deps.get_current_user`
2	Role	The user's role (`owner`, `staff`) is checked; owner-only routes return HTTP 403 for staff	`deps.require_owner`
3	Plan	Premium features (11 of them, such as the Cash Advisor, credit score and AI agent) are checked against the business's plan; otherwise HTTP 402 with an "Upgrade to Pro" message	`deps.needs`, `plans.py`
4	Business scoping and object ownership	Every query filters by the business of the signed-in user, never by an id from the request; any id in a URL is verified to belong to that business before it is read or changed	`deps.get_business`, `deps.owned`
5	Database row-level security	PostgreSQL policies on every table with a `business_id`: a request only sees rows of its own business even if a query forgets the filter. Forced on the table owner too. A start-up warning fires if the database role would skip RLS	migration `0005`, `db.py`

1.1 Who can do what

Tier	How it is identified	Allowed	Denied
Anonymous	No token	Sign up, sign in, refresh, health check	Everything else (HTTP 401 "Not authenticated")
Staff (an owner's employee, up to 10 per business)	JWT with role `staff`, tied to the owner's business	Record and view transactions, invoices, bills and customers of that business	Cash Advisor, forecast and analysis pages, credit score and loans, reports, AI agent, bank, billing, referrals, settings, team and lender access (HTTP 403 "Only the business owner can do this"). The staff menu also hides these pages.
Owner	JWT with role `owner`	Everything inside their own business; Pro features while on a Pro plan or trial	Any other business's data (HTTP 404); Pro features on the Free plan after the trial
Lender	Opaque token given by the owner, sent in `X-Lender-Token`; stored only as a SHA-256 hash	Read-only view of exactly the items the owner ticked (scope), until the expiry date	Anything not in scope, any write, any use after expiry or revocation ("Access token invalid, expired or revoked"). Every view is audited

AI agent	Runs inside the owner's session	Seven read-only tools; at most 5 steps per question	Choosing a business (the id comes from the session), writing data, sending anything: actions need the owner's confirmation in the UI
System jobs	Server-side only (daily job, seed, lender view)	An explicit database bypass flag to read across businesses for scheduled work	Never exposed to a request; request sessions always enforce RLS

1.2 Sensitive actions and their extra controls

Action	Control
Send a reminder (WhatsApp, email)	Only when the owner presses Send. A channel that is not connected is recorded, never faked as delivered. A failed send keeps the draft and shows the provider's reason
Request a lender introduction	Explicit consent tick; recorded and tracked; nothing is sent to any lender in this version
Grant lender access	Owner only, scope and expiry required, token shown once, revocable, audited
Delete account / export data	Owner only, audited; deletion removes staff and every record and cancels a subscription first
Turn on two-factor, change plan, add or remove staff	Owner only, audited
Bank connection	Owner only; the bank access token is stored encrypted and never sent to the browser
Plan changes by payment	Only from a signed Stripe webhook, never from the browser (Stripe is off in this version; billing is demo mode)

1.3 Authentication details

- Passwords: bcrypt, cost 12. The account locks for 15 minutes after 5 failed sign-ins. Rate limits on auth requests (10 per minute).
- Tokens: access JWT 15 minutes; refresh token 7 days, rotating, in an httpOnly, secure, SameSite=Strict cookie, and the refresh route also requires a custom header against CSRF.
- Optional second factor: authenticator-app code (TOTP). SMS codes are not used.

2. Known-risk list

Severity is the team's own judgement of the impact for a financial product if the risk is exploited: High, Medium, Low.

ID	Risk	Severity	What is in place now	Status / next step
R1	No independent penetration test or audit	Medium	Self-review, 264 automated tests, sandbox run with isolation checks	Planned: external review before real customer money or data
R2	Rate limiter is in memory: it is per server process and resets on restart	Medium	Limits per client address and signed-in user; account lockout is stored in the database and is not affected	Planned: Redis-backed limiter for several workers
R3	The access token is kept in the browser's `sessionStorage`, so a cross-site scripting bug could read it	Medium	Short 15-minute lifetime; React escapes output; the API sends a strict content-security-policy; the long-lived refresh token is httpOnly and not readable by scripts	Planned: a content-security-policy on the web client, and keeping the access token in memory only
R4	JWTs are signed with one shared secret (HS256); if it leaked, tokens could be forged	Medium	Secret only in server environment variables, never in git or the client; refresh tokens are also checked against a stored id and can be revoked	Planned: secret rotation procedure; asymmetric signing
R5	Free-tier LLM providers may keep or review submitted text	Medium	Only minimal fields are sent; customer and supplier names are replaced by codes; owners can choose local-only or no AI; every AI figure is checked against the engines and replaced by a template if it does not match	Mitigated; amounts and dates still reach the provider in cloud mode (documented to the owner)
R6	The AI could still phrase an explanation badly or follow an injected instruction	Low	Read-only tools, business id from the session, step limit, tool results and receipt text treated as data, owner confirms any action	Mitigated; residual wording risk accepted
R7	Uploads are not scanned for malware	Low	Type and size limits (CSV 2 MB, JPEG/PNG/WebP 5 MB); files are processed in memory and not stored or served	Planned: scanning if files are ever stored
R8	No alerting on unusual sign-in locations or bulk data access	Medium	Audit log of sign-ins (including failures), exports, lender access, reminders, plan, staff and settings changes; lockout after 5 failures	Planned: anomaly alerts
R9	No self-service password reset or email verification for owners	Medium	Owners can reset staff passwords; strong hashing and lockout	Planned: email verification and reset through a verified sender
R10	Row-level security only works if the database role is not a superuser	Medium	Application-level business scoping still applies on every query; the API logs a warning at start-up if the role would skip RLS; tested on PostgreSQL	Mitigated by the start-up check; deployment guide tells how to create a plain role
R11	Free hosting: no uptime guarantee, a single instance, no dedicated web-application firewall	Low	Host-provided TLS and basic protections; health check; stateless API	Accepted for the prototype

R12	Several integrations run in sandbox or demo mode: bank import (Plaid sandbox), billing (demo), financing offers (demo data), lender introductions (recorded only)	Low	Clearly labelled in the interface; no real money moves; Stripe webhooks are signed and verified when enabled	**Planned: production integrations with their own review**
R13	Reminders sent from a free mailbox address can land in spam or be spoofed; WhatsApp business-initiated messages need an approved template	Low	The owner reviews and sends each message; failures are shown with the provider's reason; header injection in email subjects is blocked	**Planned: a verified sending domain**
R14	No defined backup, retention or disaster-recovery policy of our own	Medium	Relies on the database host; users can export or delete their data at any time	**Planned: documented retention and restore tests**
R15	The credit score and loan figures are estimates and could be mistaken for decisions	Low	Every score shows its reasons; loan output is labelled "Estimate only, not a loan approval"	**Mitigated by labelling**
R16	Local development defaults to SQLite, which has no row-level security	Low	Production runs PostgreSQL; RLS tests run against PostgreSQL	**Accepted; documented**

3. Verification evidence

Evidence	What it shows
`docs/execution-evidence/automated-test-results.txt`	Full verbose pytest output: 264 passed, 3 skipped (the skipped ones are the opt-in PostgreSQL row-level-security tests). Includes access control (`test_access.py`), lender access, 2FA, secret cleaning, messaging and notifications
`docs/execution-evidence/sandbox-run-log.txt`	24 timestamped API calls with the demo account: wrong password refused (401), no token refused (401), a second throw-away account cannot read a demo customer (404) and sees none of the demo data, security headers present, reminders drafted but not sent, the AI answer is grounded in a tool result
`tests/test_rls_postgres.py`	Row-level security checked against a real PostgreSQL database (set `RLS_TEST_URL`)
`docs/manual-test-cases.md`	Manual test cases, including staff limits (H6, H7), lender access (H9 to H11), two-factor (H12, H13) and isolation (I1, I2)
Demo video	The product running end to end